

# Electromagnetism Homework #5

Problems marked with [\*] are potential sources for quiz questions.

## [\*] Problem 1: P-Polarization (Parallel Polarization)

The reflection and transmission coefficients for waves polarized *parallel* to the plane of incidence are:

$$R_p \equiv \frac{I_R}{I_I} = \left( \frac{E_{0,R}}{E_{0,I}} \right)^2 = \left( \frac{\alpha - \beta}{\alpha + \beta} \right)^2$$

$$T_p \equiv \frac{I_T}{I_I} = \frac{\epsilon_2 v_2}{\epsilon_1 v_1} \left( \frac{E_{0,T}}{E_{0,I}} \right) \frac{\cos \theta_T}{\cos \theta_I} = \alpha \beta \left( \frac{2}{\alpha + \beta} \right)^2$$

with,

$$\beta = \frac{\mu_1 n_2}{\mu_2 n_1} \quad \text{and} \quad \alpha = \frac{\cos \theta_T}{\cos \theta_I} = \frac{\sqrt{1 - \left( \frac{n_1}{n_2} \sin \theta_I \right)^2}}{\cos \theta_I}$$

Plot  $R_p$  and  $T_p$  vs  $\theta_I$  for  $0 \leq \theta_i < 90^\circ$  for an air-diamond interface.  $n_{\text{air}} = 1$ ,  $n_{\text{diamond}} = 2.42$ . Assume  $\mu_{\text{air}} = \mu_{\text{diamond}} = \mu_0$ . Calculate and mark Brewster's angle where  $R_p = 0$ ,  $\theta_B$ , for these materials and indicate it on the plot.

# Problem 1 Solution:

Brewster's angle describes the angle of incidence where light with a certain polarization is perfectly transmitted through a transparent **dielectric** surface with no reflection. In this case  $R_p = 0$ .

Assuming  $\mu_1 = \mu_2 = \mu_0$ , the calculations of Brewster's angle can be done:

$$\theta_B = \arctan\left(\frac{n_2}{n_1}\right)$$

Given  $n_{air} = 1$  and  $n_{diamond} = 2.42$ :

$$\theta_B = \arctan\left(\frac{2.42}{1}\right)$$

$$\theta_B \approx 67.54^\circ$$

```
In [24]: import numpy as np
import matplotlib.pyplot as plt

# Given constants
n1 = 1.0      # n_air
n2 = 2.42     # n_diamond
mu_ratio = 1.0 # mu1/mu2 = 1 since both are mu_0

# Array of incident angles from 0 to just under 90 degrees (in radians)
theta_I_deg = np.linspace(0, 89.99, 500)
theta_I = np.radians(theta_I_deg)

# Calculate Beta
beta = mu_ratio * (n2 / n1)

# Calculate Alpha
# Using the given formula: alpha = sqrt(1 - (n1/n2 * sin(theta_I))^2) / cos(theta_I)
numerator = np.sqrt(1 - (n1 / n2 * np.sin(theta_I))**2)
denominator = np.cos(theta_I)
alpha = numerator / denominator

# Calculate Reflection (Rp) and Transmission (Tp) coefficients
Rp = ((alpha - beta) / (alpha + beta))**2
Tp = alpha * beta * (2 / (alpha + beta))**2

# Calculate Brewster's Angle
theta_B_rad = np.arctan(n2 / n1)
theta_B_deg = np.degrees(theta_B_rad)

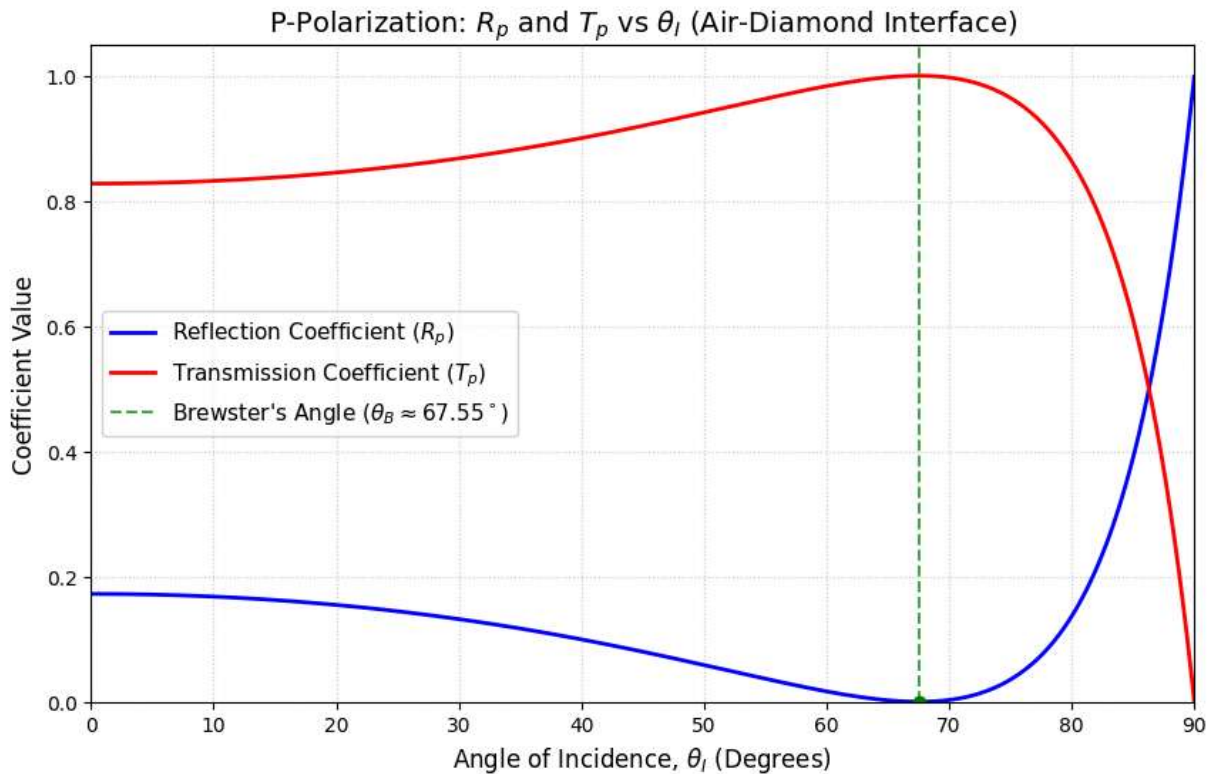
# Plotting
plt.figure(figsize=(10, 6))
plt.plot(theta_I_deg, Rp, label=r'Reflection Coefficient ($R_p$)', color='blue', li
plt.plot(theta_I_deg, Tp, label=r'Transmission Coefficient ($T_p$)', color='red', l

# Mark Brewster's Angle
```

```
plt.axvline(x=theta_B_deg, color='green', linestyle='--', alpha=0.7, label=rf"Brews
plt.plot(theta_B_deg, 0, 'go') # Point where  $R_p = 0$ 

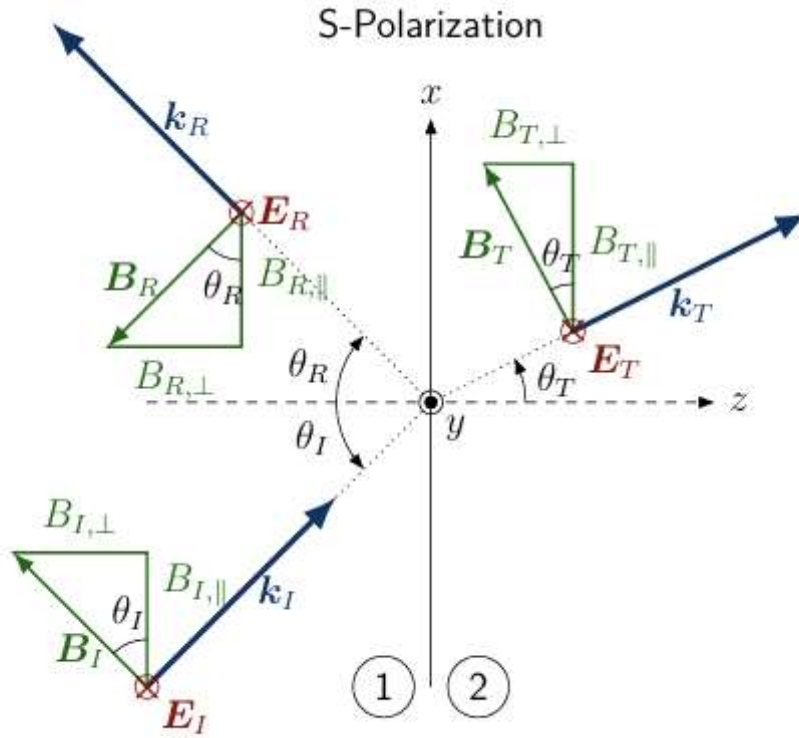
# Formatting the plot
plt.title(r'P-Polarization:  $R_p$  and  $T_p$  vs  $\theta_i$  (Air-Diamond Interface)',
plt.xlabel(r'Angle of Incidence,  $\theta_i$  (Degrees)', fontsize=12)
plt.ylabel('Coefficient Value', fontsize=12)
plt.xlim(0, 90)
plt.ylim(0, 1.05)
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend(fontsize=11)

plt.show()
```



## [\*] Problem 2: S-Polarization (Perpendicular Polarization)

Analyze the case of polarization *perpendicular* to the plane of incidence (that is, electric fields in the  $y$  direction) as illustrated in the figure below. Do the following:



(a) Impose the boundary conditions for linear media:

$$\epsilon_1 E_1^\perp = \epsilon_2 E_2^\perp \quad \mathbf{E}_1^\parallel = \mathbf{E}_2^\parallel$$

$$B_1^\perp = B_2^\perp \quad \frac{1}{\mu_1} \mathbf{B}_1^\parallel = \frac{1}{\mu_2} \mathbf{B}_2^\parallel$$

and obtain the general Fresnel equations for  $E_{0,R}$  and  $E_{0,T}$ . You should find:

$$\frac{E_{0,R}}{E_{0,I}} = \frac{1 - \alpha\beta}{1 + \alpha\beta}, \quad \frac{E_{0,T}}{E_{0,I}} = \frac{2}{1 + \alpha\beta}$$

with,

$$\alpha = \frac{\cos \theta_T}{\cos \theta_I} = \frac{\sqrt{1 - \left(\frac{n_1}{n_2} \sin \theta_I\right)^2}}{\cos \theta_I}, \quad \beta = \frac{\mu_1 n_2}{\mu_2 n_1}$$

(b) Plot  $E_{0,R}/E_{0,I}$  and  $E_{0,T}/E_{0,I}$  vs  $\theta_i$  for the case  $\beta = n_2/n_1 = 1.5$  and  $\mu_1 = \mu_2 = \mu_0$ . (Note for this  $\beta$  the reflected wave is *always* 180° out of phase.)

(c) Show that for nonmagnetic media ( $\mu_1 = \mu_2 = \mu_0$ ) there is no Brewster angle for S-polarization:  $E_{0,R}$  is *never* zero (unless, of course,  $n_1 = n_2$ , in which case the two media are optically indistinguishable).



(d) Confirm that your Fresnel equations reduce to the normal-incidence reflection/transmission expressions when  $\theta_I = 90^\circ$ .

---

(e) Compute the reflection and transmission coefficients, and check that they add up to 1. You should find:

$$R_s = \left( \frac{1 - \alpha\beta}{1 + \alpha\beta} \right)^2, \quad T_s = \alpha\beta \left( \frac{2}{1 + \alpha\beta} \right)^2$$

with  $\alpha$  and  $\beta$  defined earlier.

---

(f) Plot  $R_s$  and  $T_s$  vs  $\theta_I$  for an air-diamond interface.  $n_{\text{air}} = 1$ ,  $n_{\text{diamond}} = 2.42$ . Assume  $\mu_{\text{air}} = \mu_{\text{diamond}} = \mu_0$ .

## Problem 2 Solution:

```
In [25]: # Constants
n1 = 1.0      # n_air
n2 = 2.42     # n_diamond
mu_ratio = 1.0 # mu1/mu2

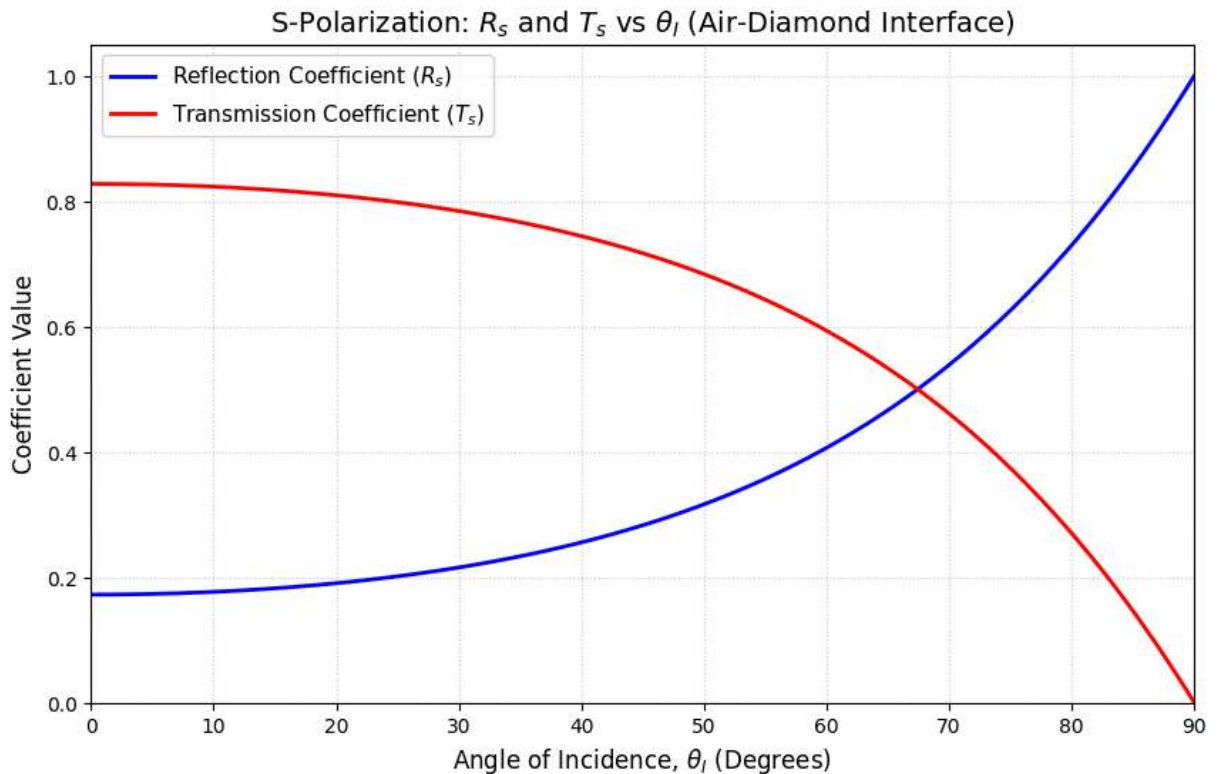
# Incident angles
theta_I_deg = np.linspace(0, 89.99, 500)
theta_I = np.radians(theta_I_deg)

# Calculate Beta and Alpha
beta = mu_ratio * (n2 / n1)
numerator = np.sqrt(1 - (n1 / n2 * np.sin(theta_I))**2)
denominator = np.cos(theta_I)
alpha = numerator / denominator

# Calculate S-Polarization Coefficients
Rs = ((1 - alpha * beta) / (1 + alpha * beta))**2
Ts = alpha * beta * (2 / (1 + alpha * beta))**2

# Plotting
plt.figure(figsize=(10, 6))
plt.plot(theta_I_deg, Rs, label=r'Reflection Coefficient ($R_s$)', color='blue', li
plt.plot(theta_I_deg, Ts, label=r'Transmission Coefficient ($T_s$)', color='red', l

# Formatting
plt.title(r'S-Polarization: $R_s$ and $T_s$ vs $\theta_I$ (Air-Diamond Interface)',
plt.xlabel(r'Angle of Incidence, $\theta_I$ (Degrees)', fontsize=12)
plt.ylabel('Coefficient Value', fontsize=12)
plt.xlim(0, 90)
plt.ylim(0, 1.05)
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend(fontsize=11)
plt.show()
```



## Problem 3: Index of Refraction of Water

In the homework folder there is a file named `index-of-refraction-and-absorption-of-water.csv` which was copied from the table found here:

[https://en.wikipedia.org/wiki/Optical\\_properties\\_of\\_water\\_and\\_ice](https://en.wikipedia.org/wiki/Optical_properties_of_water_and_ice). Do the following:

**(a)** Make a plot of the index of refraction vs. wavelength in the range 200 to 1000-nm. Label the axes and give the plot an appropriate title.

**(b)** Fit the data to Cauchy's formula for the index of refraction,

$$n(\lambda) = A + \frac{B}{\lambda^2} + \frac{C}{\lambda^4},$$

and determine the best-fit values for  $A$ ,  $B$ , and  $C$ . Include the fit on the plot with the data. Refer to the curve-fitting tutorial in the homework folder if needed.

**(c)** Determine a range of wavelengths that correspond to "red" and "blue" light and find a representative value of the index of refraction for each color.

## Problem 3 Solution:

```

In [26]: import pandas as pd
         from scipy.optimize import curve_fit

         # File path
         csv_path = "index-of-refraction-and-absorption-of-water.csv"

         # Read CSV (header has spaces/newline quirks, so clean names after reading)
         df = pd.read_csv(csv_path)
         df.columns = [c.strip() for c in df.columns]

         # Find wavelength and n columns robustly
         wl_col = [c for c in df.columns if "Wavelength" in c][0]
         n_col = [c for c in df.columns if c.strip() == "n"][0]

         # Keep only needed columns and force numeric
         data = df[[wl_col, n_col]].copy()
         data[wl_col] = pd.to_numeric(data[wl_col], errors="coerce")
         data[n_col] = pd.to_numeric(data[n_col], errors="coerce")
         data = data.dropna()

         # Convert wavelength to nm for plotting/filtering
         data["wavelength_um"] = data[wl_col]
         data["wavelength_nm"] = 1000.0 * data["wavelength_um"]
         data["n"] = data[n_col]

         # Problem asks 200 to 1000 nm
         fit_data = data[(data["wavelength_nm"] >= 200) & (data["wavelength_nm"] <= 1000)].c

         lam_um = fit_data["wavelength_um"].to_numpy()
         n_vals = fit_data["n"].to_numpy()

         # Cauchy fit:  $n(\lambda) = A + B/\lambda^2 + C/\lambda^4$ 
         # Use linear least squares on transformed basis (no scipy needed)
         X = np.column_stack([
             np.ones_like(lam_um),
             1.0 / (lam_um**2),
             1.0 / (lam_um**4)
         ])
         A, B, C = np.linalg.lstsq(X, n_vals, rcond=None)[0]

         def cauchy(lam_um):
             return A + B/(lam_um**2) + C/(lam_um**4)

         # Smooth curve for plotting
         lam_plot_um = np.linspace(0.2, 1.0, 500)
         lam_plot_nm = 1000.0 * lam_plot_um
         n_plot = cauchy(lam_plot_um)

         # Plot data + fit
         plt.figure(figsize=(9, 5.5))
         plt.scatter(fit_data["wavelength_nm"], fit_data["n"], s=26, label="Water data (200-1000 nm)")
         plt.plot(lam_plot_nm, n_plot, linewidth=2.2, label="Cauchy fit")
         plt.xlabel("Wavelength (nm)")
         plt.ylabel("Refractive index, n")
         plt.title("Index of Refraction of Water vs Wavelength (200-1000 nm)")

```

```

plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()

print(f"Solutions to part (b):")
print(f"Cauchy fit coefficients:")
print(f"A = {A:.8f}")
print(f"B = {B:.8f} (um^2)")
print(f"C = {C:.8f} (um^4)")

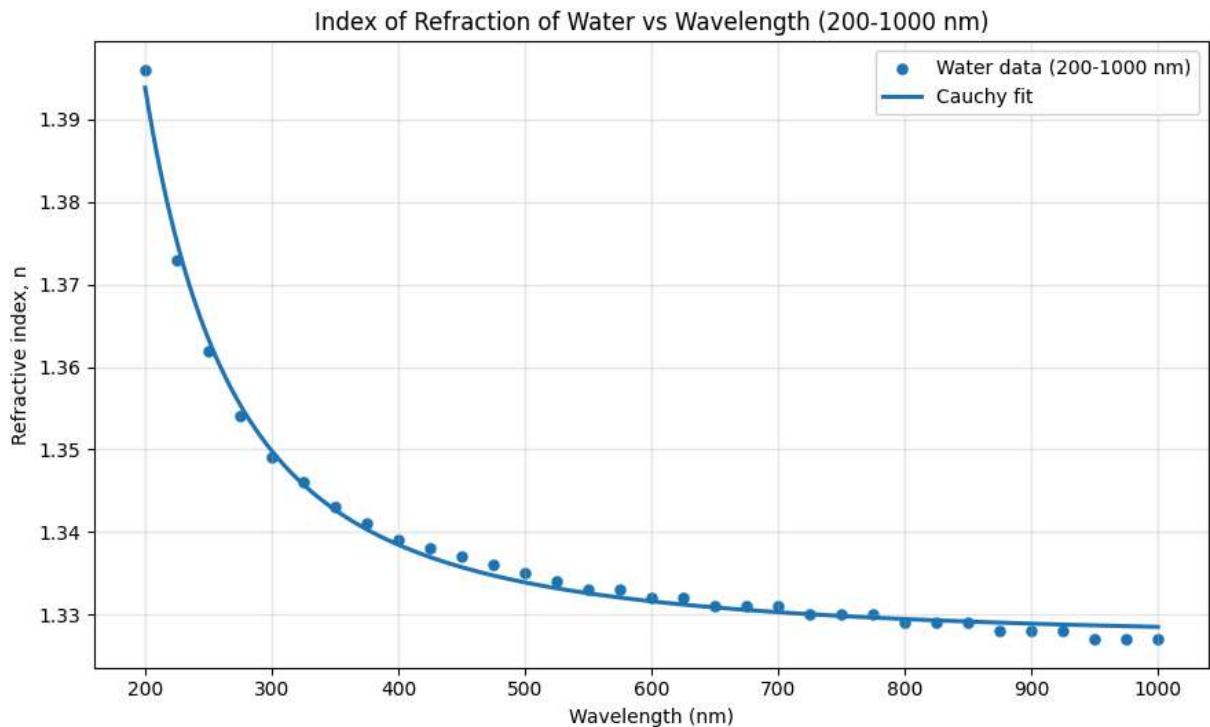
# Part (c): representative red/blue ranges and values
red_range = fit_data[(fit_data["wavelength_nm"] >= 620) & (fit_data["wavelength_nm"] <= 750)]
blue_range = fit_data[(fit_data["wavelength_nm"] >= 450) & (fit_data["wavelength_nm"] <= 495)]

n_red_rep = red_range["n"].mean()
n_blue_rep = blue_range["n"].mean()

print(f"\nSolutions to part (c):")
print("Representative wavelength ranges:")
print("Red: 620-750 nm")
print("Blue: 450-495 nm")
print(f"Representative n_red (mean over range) = {n_red_rep:.5f}")
print(f"Representative n_blue (mean over range) = {n_blue_rep:.5f}")

# Optional: values at specific wavelengths often used in optics
n_700 = cauchy(0.700)
n_450 = cauchy(0.450)
print(f"\nCauchy n(700 nm) = {n_700:.5f}")
print(f"Cauchy n(450 nm) = {n_450:.5f}")

```

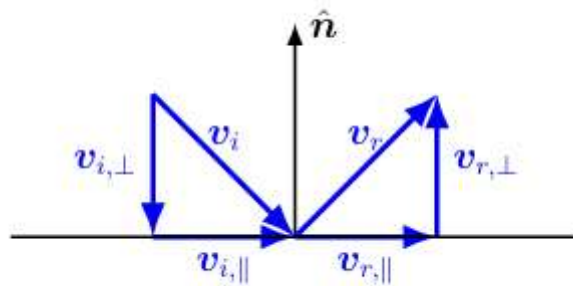


Solutions to part (b):  
 Cauchy fit coefficients:  
 $A = 1.32686543$   
 $B = 0.00157624 \text{ (um}^2\text{)}$   
 $C = 0.00004414 \text{ (um}^4\text{)}$

Solutions to part (c):  
 Representative wavelength ranges:  
 Red: 620-750 nm  
 Blue: 450-495 nm  
 Representative  $n_{\text{red}}$  (mean over range) = 1.33083  
 Representative  $n_{\text{blue}}$  (mean over range) = 1.33650  
  
 Cauchy  $n(700 \text{ nm}) = 1.33027$   
 Cauchy  $n(450 \text{ nm}) = 1.33573$

## Problem 4: Reflection Formula Derivation

Derive the Reflection Formula: A light ray with direction vector  $\mathbf{v}_i$  strikes a smooth surface at a point where the unit normal vector is  $\hat{\mathbf{n}}$ . The reflected ray direction is denoted  $\mathbf{v}_r$ . See illustration below.




---

**(a)** Decompose  $\mathbf{v}_i$  into components parallel and perpendicular to the surface.

---

**(b)** Show that the component perpendicular to the surface can be written:  $\mathbf{v}_{i,\perp} = (\mathbf{v}_i \cdot \hat{\mathbf{n}})\hat{\mathbf{n}}$ .

---

**(c)** Argue that  $\mathbf{v}_{i,\parallel} = \mathbf{v}_i - \mathbf{v}_{i,\perp}$ .

---

**(d)** To reflect the ray, the parallel component,  $\mathbf{v}_{r,\parallel}$ , remains unchanged, while the perpendicular component,  $\mathbf{v}_{r,\perp}$ , reverses direction. How would you write this as a vector equation?

---

**(e)** Using the previous step, show that the reflection formula can be written:

$$\mathbf{v}_r = \mathbf{v}_i - 2(\mathbf{v}_i \cdot \hat{\mathbf{n}})\hat{\mathbf{n}}$$


---

(f) Apply your result to compute the reflected direction of the vector  $\mathbf{v}_i = 0.8\hat{x} - 0.6\hat{y}$  off a surface with normal  $\hat{n} = 0\hat{x} + 1\hat{y}$ . Does your result make sense?

## Problem 4 Solution:

### Part (a):

This is pretty simple:

$$\mathbf{v}_i = \mathbf{v}_{i,\perp} + \mathbf{v}_{i,\parallel}$$

### Part (b):

After some research and remembrance of how dot products and scalar projections work, I found that the perpendicular component is just the projection of  $\mathbf{v}_i$  onto  $\hat{n}$ . By definition of the dot product, the scalar projection is  $\mathbf{v}_i \cdot \hat{n}$ , so the vector component is:

$$\mathbf{v}_{i,\perp} = (\mathbf{v}_i \cdot \hat{n})\hat{n}$$

### Part (c):

This is just a rearrangement of part (a). Its simple math.

### Part (d):

$$\mathbf{v}_r = \mathbf{v}_{r,\parallel} + \mathbf{v}_{r,\perp} = \mathbf{v}_{r,\parallel} - \mathbf{v}_{i,\perp}$$

### Part (e):

To do this we just have to plug in part b and part c to part d:

$$\mathbf{v}_r = (\mathbf{v}_i - \mathbf{v}_{i,\perp}) - \mathbf{v}_{i,\perp} = \mathbf{v}_i - 2\mathbf{v}_{i,\perp} = \mathbf{v}_i - 2(\mathbf{v}_i \cdot \hat{n})\hat{n}$$

### Part (f):

If we take the given functions/ values of  $\mathbf{v}_i$  and  $\hat{n}$ , then we can plug them into the equation we got in part (e).

$$\mathbf{v}_i \cdot \hat{n} = -0.6$$

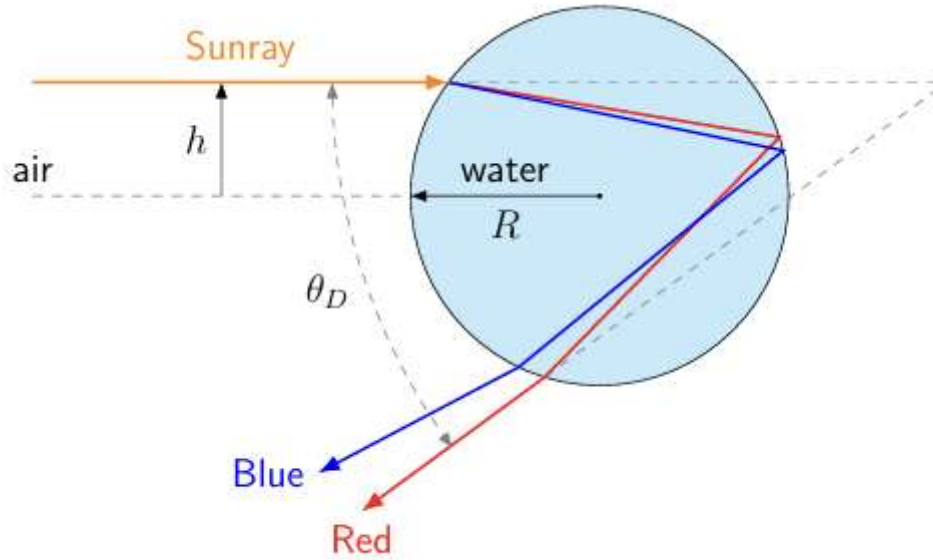
and,

$$\mathbf{v}_r = [0.8, -0.6] - 2(-0.6)[0, 1] = [0.8, 0.6]$$

This makes sense because the x-component is unchanged, and the y-component flipped from -0.6 to +0.6.

## Problem 5: The Rainbow

Here we work through the multiple reflections and refractions that give rise to the rainbow. Consider a ray of sunlight incident from the left a distance  $h$  above the  $x$ -axis on a spherical drop of water with radius 1 centered at the origin such that  $0 < h < 1$ , as illustrated in Figure 1. The index of refraction for air,  $n_{\text{air}} = 1$ , and use your index of refraction values for red and blue light from a previous problem.



**Figure 1 NOTE:** the indices of refraction are  $n_{\text{red}} = 1.3$  for red light (700nm) and  $n_{\text{blue}} = 1.4$  for blue light (400nm) in this figure for illustration purposes only.

**(a) Point of Entry and Surface Normal:** The entry point on the circle is,

$$\mathbf{r}_1 = -(\sqrt{R^2 - h^2})\hat{x} + h\hat{y}$$

The outward unit normal at this point is:

$$\hat{n} = \frac{\mathbf{r}_1}{R} = -\frac{\sqrt{R^2 - h^2}}{R}\hat{x} + \frac{h}{R}\hat{y}$$

Pick a value of  $h$  in the range  $0 < h < 1$  and calculate the vectors  $\mathbf{r}_1$  and  $\hat{n}$  for  $R = 1$ .

**(b) Incident Angle & Transmitted Angle:** The incoming sunray is in the direction,

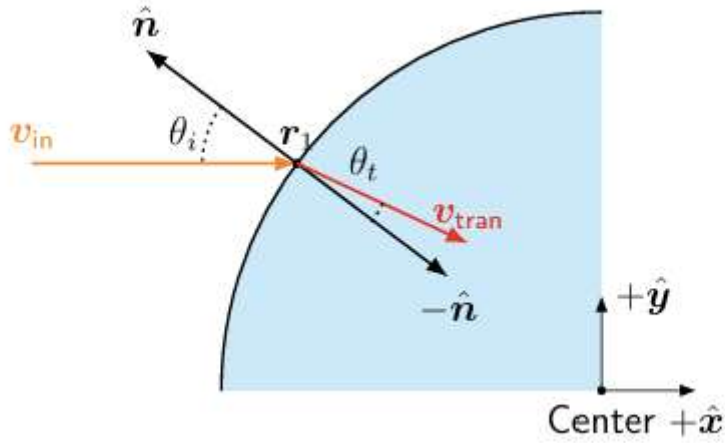
$$\mathbf{v}_{\text{in}} = +1\hat{x} + 0\hat{y}$$

Determine the angle of incidence,  $\theta_i$ , by computing the angle between the incoming ray direction (from the Sun) and the outward normal using the dot product formula:

$$\theta_i = \arccos(-\mathbf{v}_{\text{in}} \cdot \hat{n})$$

(Note: we use the negative dot product since the ray comes from outside the drop.)

Calculate  $\theta_i$  for your choice of  $h$ .



Use Snell's Law to calculate the transmission angle:

$$n_{\text{air}} \sin \theta_i = n_{\text{red}} \sin \theta_t \quad \Rightarrow \quad \theta_t = \arcsin\left(\frac{n_{\text{air}}}{n_{\text{red}}} \sin \theta_i\right)$$

Compute  $\theta_t$  for red light for your value of  $\theta_i$ .

---

**(c) Direction of Refracted Ray:** Rotate the inward-pointing surface normal by the transmission angle to construct the new direction vector for the transmitted ray inside the drop. The inward normal is  $-\hat{n}$ . We rotate this vector **counter-clockwise** by  $\theta_t$  to obtain the transmitted ray direction using matrix multiplication:

$$\mathbf{v}_{\text{trans}} = \mathcal{R}(\theta_t)(-\hat{n})$$

with the 2D rotation matrix:

$$\mathcal{R}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

and

$$-\hat{n} = \frac{\sqrt{R^2 - h^2}}{R} \hat{x} - \frac{h}{R} \hat{y} = \begin{bmatrix} \sqrt{R^2 - h^2}/R \\ -h/R \end{bmatrix}$$

Compute  $\mathbf{v}_{\text{trans}}$  using your calculated value of  $\theta_t$ .

---

**(d) First Internal Reflection Location:** Recall  $\mathbf{r}_1$  is the point where the ray enters the raindrop. Let  $\hat{t}$  be the unit vector pointing in the direction of the ray after it has refracted inside the drop, in the direction of  $\mathbf{v}_{\text{trans}}$ :

$$\hat{t} = \frac{\mathbf{v}_{\text{trans}}}{\|\mathbf{v}_{\text{trans}}\|}$$



Compute  $\hat{t}$  from  $\mathbf{v}_{\text{trans}}$ . We now wish to find the point  $\mathbf{r}_2$  where this ray next intersects the boundary of the raindrop. Since the drop is a circle of radius  $R = 1$  centered at the origin, the ray inside follows a parametric path of the form:

$$\mathbf{x}(s) = \mathbf{r}_1 + s\hat{t}, \quad s > 0$$

The intersection point is on the circle so  $\|\mathbf{x}(s)\|^2 = R^2 (= 1)$ . Substituting and expanding gives a quadratic equation in  $s$ :

$$s^2 + 2(\mathbf{r}_1 \cdot \hat{t})s + (\|\mathbf{r}_1\|^2 - R^2) = 0$$

Solving this gives two values of  $s$ :

$$s = \frac{-2(\mathbf{r}_1 \cdot \hat{t}) \pm \sqrt{[2(\mathbf{r}_1 \cdot \hat{t})]^2 - 4(\|\mathbf{r}_1\|^2 - R^2)}}{2}$$

with  $R = 1$ . The **smaller root**,  $s_1$ , corresponds to the original point  $\mathbf{r}_1$ . The **larger root**,  $s_2$ , gives the location of the second intersection:

$$\mathbf{r}_2 = \mathbf{r}_1 + s_2\hat{t}$$

Compute the roots  $s_1$  and  $s_2$ . Use  $s_2$  to determine  $\mathbf{r}_2$ .

**(e) Internal Reflection:** At this new point  $\mathbf{r}_2$ , the **interior** surface normal is:

$$\hat{n}_2 = -\frac{\mathbf{r}_2}{\|\mathbf{r}_2\|} = -\frac{\mathbf{r}_2}{R}, \quad \text{with } R = 1$$

Compute  $\hat{n}_2$  at  $\mathbf{r}_2$ . We then apply the vector reflection formula (previous problem) to compute the new direction:

$$\hat{r} = \hat{t} - 2(\hat{t} \cdot \hat{n}_2)\hat{n}_2$$

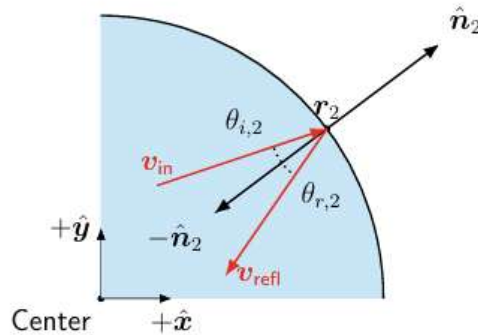


Figure 3: Illustration of internal reflection at  $\mathbf{r}_2$ .  $\theta_{i,2} = \theta_{r,2}$  from the Law of Reflection.

This reflected direction  $\hat{r}$  is the direction in which the ray will travel,  $\mathbf{v}_{\text{refl}}$ , toward the opposite wall of the drop for potential refraction and exit.

---

**(f) Exit Refraction:** Let  $\mathbf{r}_3$  be the third intersection point on the circle. Find this point via a similar method as in part (d). At this point, compute the new angle of incidence  $\theta_{i,3}$  relative to the inward surface normal  $-\hat{\mathbf{n}}_3 = -\mathbf{r}_3/R$ , with  $R = 1$ :

$$\theta_{i,3} = \arccos(-\mathbf{v}_{\text{refl}} \cdot (-\hat{\mathbf{n}}_3))$$

where  $\mathbf{v}_{\text{refl}} = \mathbf{v}_{\text{in}}$  in Figure 4.

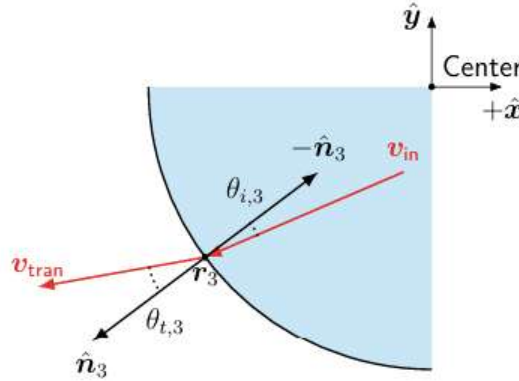


Figure 4: Vectors relevant to incident and refracted rays from inside the raindrop to outside.

Then use Snell's Law to compute the transmission angle:

$$\theta_{t,3} = \arcsin\left(\frac{n_{\text{red}}}{n_{\text{air}}}\sin\theta_{i,3}\right)$$

The final exit direction is computed by rotating the normal vector  $\hat{\mathbf{n}}_3$  **clockwise** by  $\theta_{t,3}$  via matrix multiplication:

$$\mathbf{v}_{\text{exit}} = \mathcal{R}(-\theta_{t,3})\hat{\mathbf{n}}_3$$

Note: Clockwise rotations are negative angles. Counter-clockwise rotations are positive angles. Determine  $\theta_{t,3}$  and  $\mathbf{v}_{\text{exit}}$ .

---

**(g) Scattering Angle (Rainbow Angle):** The observable scattering angle between the incident sunlight direction and the exiting ray is called the **deviation angle**, denoted  $\Theta_D$ . We define the deviation angle as the angle between the reversed incident direction ( $-\mathbf{v}_{\text{incident}}$ ) and the final exit direction  $\mathbf{v}_{\text{exit}}$ :

$$\Theta_D = \cos^{-1}\left(\frac{(-\mathbf{v}_{\text{incident}}) \cdot \mathbf{v}_{\text{exit}}}{\|\mathbf{v}_{\text{incident}}\| \cdot \|\mathbf{v}_{\text{exit}}\|}\right)$$

Since both vectors are unit length in our construction, this simplifies to:

$$\Theta_D = \cos^{-1}(-\mathbf{v}_{\text{incident}} \cdot \mathbf{v}_{\text{exit}})$$

(h) Now that you have a sequence of steps to go from a choice of  $h$  (subject to  $0 < h < 1$ ) to  $\mathbf{v}_{\text{exit}}$ , automate the process and vary the entry height  $h$  over the range  $0 < h < 1$  and compute the deviation angle  $\Theta_D$  as a function of  $h$ . Plot  $\Theta(h)$ , and determine the extremum of the deviation function for red light. Repeat for blue light. Which is deviated more?

This maximum in deviation angle defines the angle at which a concentration of rays exit the drop, leading to the bright arc of the primary rainbow. For red light in water, this angle is typically close to  $42^\circ$  as illustrated in Figure 5.

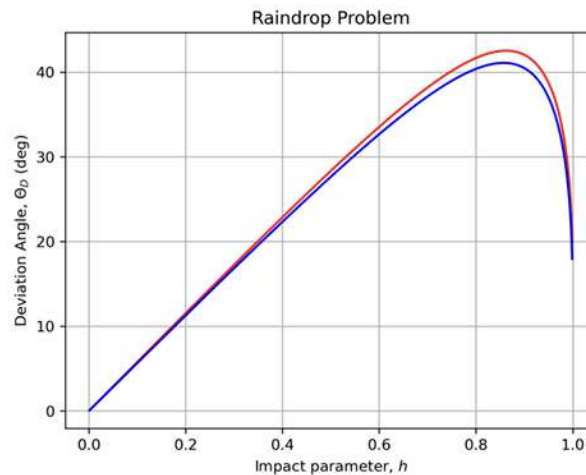


Figure 5: Plot of the deviation angle,  $\Theta_D$ , vs the impact parameter,  $h$ , for rays of sunlight hitting a spherical raindrop for two indices of refraction:  $n_{\text{red}} = 1.33$  and  $n_{\text{blue}} = 1.34$ . Notice the maximum value for red light is  $\sim 42^\circ$ .

## Problem 5 Solution:

```
In [27]: # Part (a):
h, R = 0.6, 1.0
r1 = np.array([-np.sqrt(R**2 - h**2), h])
n_hat = r1 / R # already unit since R=1

print(f"r1 = {r1}")
print(f"n_hat = {n_hat}")
```

```
r1 = [-0.8  0.6]
n_hat = [-0.8  0.6]
```

```
In [28]: # Part (b):
h = 0.6
n_air = 1.0
n_red = 1.3
v_in = np.array([1.0, 0.0])
theta_i = np.arccos(-np.dot(v_in, n_hat))
theta_t = np.arcsin((n_air / n_red) * np.sin(theta_i))
```

```
print(f"theta_i = {np.degrees(theta_i):.2f} degrees")
print(f"theta_t = {np.degrees(theta_t):.2f} degrees")
```

theta\_i = 36.87 degrees

theta\_t = 27.49 degrees

```
In [29]: # Part (c):
def rotation_matrix(theta):
    return np.array([[np.cos(theta), -np.sin(theta)],
                     [np.sin(theta), np.cos(theta)]])

v_trans = rotation_matrix(theta_t) @ (-n_hat)

print(f"v_trans = {v_trans}")
```

v\_trans = [ 0.98661924 -0.16304135]

```
In [30]: # Part (d):
t_hat = v_trans / np.linalg.norm(v_trans)
a_coef = 1.0
b_coef = 2 * np.dot(r1, t_hat)
c_coef = np.dot(r1, r1) - R**2
discriminant = b_coef**2 - 4*c_coef
s1 = (-b_coef - np.sqrt(discriminant)) / 2
s2 = (-b_coef + np.sqrt(discriminant)) / 2
r2 = r1 + s2 * t_hat

print(f"r2 = {r2}")
print(f"s1 = {s1:.2f}, s2 = {s2:.2f}")
```

r2 = [0.95049971 0.31072545]

s1 = 0.00, s2 = 1.77

```
In [31]: #Part (e):
n2_hat = -r2 / np.linalg.norm(r2)
v_refl = t_hat - 2 * np.dot(t_hat, n2_hat) * n2_hat

print(f"n2_hat = {n2_hat}")
print(f"v_refl = {v_refl}")
```

n2\_hat = [-0.95049971 -0.31072545]

v\_refl = [-0.69979575 -0.714343 ]

```
In [32]: # Part (f):
r_hat = v_refl / np.linalg.norm(v_refl)
b2 = 2 * np.dot(r2, r_hat)
c2 = np.dot(r2, r2) - R**2
s3 = (-b2 + np.sqrt(b2**2 - 4*c2)) / 2
r3 = r2 + s3 * r_hat

n3_hat = r3 / np.linalg.norm(r3) # outward normal at r3
inward_n3 = -n3_hat
theta_i3 = np.arccos(np.dot(-v_refl, inward_n3))
theta_t3 = np.arcsin((n_red / n_air) * np.sin(theta_i3))
v_exit = rotation_matrix(-theta_t3) @ n3_hat

print(f"r3 = {r3}")
print(f"n3_hat = {n3_hat}")
```

```

print(f"theta_i3 = {np.degrees(theta_i3):.2f} degrees")
print(f"theta_t3 = {np.degrees(theta_t3):.2f} degrees")
print(f"v_exit = {v_exit}")

```

```

r3 = [-0.29110617 -0.95669075]
n3_hat = [-0.29110617 -0.95669075]
theta_i3 = 27.49 degrees
theta_t3 = 36.87 degrees
v_exit = [-0.80689939 -0.5906889 ]

```

```

In [33]: #part (g):
        Theta_D = np.degrees(np.arccos(-np.dot(v_in, v_exit)))

        print(f"Deflection angle Theta_D = {Theta_D:.2f} degrees")

```

Deflection angle Theta\_D = 36.21 degrees

```

In [34]: # Part (h): Automate parts (a)-(g) into a function and sweep h over (0, 1)
        # Use n values from Problem 3 Cauchy fit (n_red_rep, n_blue_rep)
        n_air_rain = 1.0
        n_red_rain = n_red_rep # ~1.331 at 700 nm
        n_blue_rain = n_blue_rep # ~1.337 at 450 nm

        def raindrop_deviation(h, n_water, R=1.0):
            """
            Trace a sunray entering a spherical raindrop at height h.
            Returns the deviation angle Theta_D in degrees (parts a-g in one function).
            """

            # (a) Entry point and outward unit normal
            r1 = np.array([-np.sqrt(R**2 - h**2), h])
            n_hat_1 = r1 / R

            # (b) Incident angle and transmitted angle (Snell's Law, air -> water)
            v_in = np.array([1.0, 0.0])
            theta_i = np.arccos(np.clip(-np.dot(v_in, n_hat_1), -1.0, 1.0))
            sin_t = (n_air_rain / n_water) * np.sin(theta_i)
            if abs(sin_t) > 1.0:
                return np.nan
            theta_t = np.arcsin(sin_t)

            # Rotation matrix helper
            def R_mat(angle):
                return np.array([[np.cos(angle), -np.sin(angle)],
                                  [np.sin(angle), np.cos(angle)]])

            # (c) Refracted ray direction inside drop: rotate -n_hat CCW by theta_t
            v_trans = R_mat(theta_t) @ (-n_hat_1)

            # (d) Find r2 - next intersection of ray with drop boundary
            t_hat = v_trans / np.linalg.norm(v_trans)
            b = 2 * np.dot(r1, t_hat)
            c = np.dot(r1, r1) - R**2
            s2 = (-b + np.sqrt(b**2 - 4*c)) / 2 # Larger root
            r2 = r1 + s2 * t_hat

            # (e) Internal reflection at r2 using reflection formula from Problem 4
            n2_hat = -r2 / np.linalg.norm(r2) # interior normal

```

```

v_refl = t_hat - 2 * np.dot(t_hat, n2_hat) * n2_hat

# (f) Find r3 - exit intersection; compute exit refraction (water -> air)
r_hat = v_refl / np.linalg.norm(v_refl)
b3 = 2 * np.dot(r2, r_hat)
c3 = np.dot(r2, r2) - R**2
s3 = (-b3 + np.sqrt(b3**2 - 4*c3)) / 2
r3 = r2 + s3 * r_hat
n3_hat = r3 / np.linalg.norm(r3)      # outward normal at exit point

theta_i3 = np.arccos(np.clip(np.dot(-v_refl, -n3_hat), -1.0, 1.0))
sin_t3 = (n_water / n_air_rain) * np.sin(theta_i3)
if abs(sin_t3) > 1.0:
    return np.nan                    # total internal reflection - no exit
theta_t3 = np.arcsin(sin_t3)
v_exit = R_mat(-theta_t3) @ n3_hat   # rotate n3_hat CW by theta_t3

# (g) Deviation angle between -v_in and v_exit
return np.degrees(np.arccos(np.clip(-np.dot(v_in, v_exit), -1.0, 1.0)))

# Sweep h from just above 0 to just below 1
h_vals = np.linspace(0.01, 0.999, 1000)
Theta_red = np.array([raindrop_deviation(h, n_red_rain) for h in h_vals])
Theta_blue = np.array([raindrop_deviation(h, n_blue_rain) for h in h_vals])

# Find rainbow angle = maximum deviation
idx_red = np.nanargmax(Theta_red)
idx_blue = np.nanargmax(Theta_blue)

print(f"Red light rainbow angle: {Theta_red[idx_red]:.2f} deg at h = {h_vals[idx_red]:.2f} (n = {n_red_rain})")
print(f"Blue light rainbow angle: {Theta_blue[idx_blue]:.2f} deg at h = {h_vals[idx_blue]:.2f} (n = {n_blue_rain})")
print(f"\nBlue light is deviated {'more' if Theta_blue[idx_blue] > Theta_red[idx_red] else 'less'} than red light.")

# Plot Theta_D(h) for both colors
plt.figure(figsize=(9, 5.5))
plt.plot(h_vals, Theta_red, color='red', linewidth=2, label=f'Red (n = {n_red_rain})')
plt.plot(h_vals, Theta_blue, color='blue', linewidth=2, label=f'Blue (n = {n_blue_rain})')
plt.axvline(h_vals[idx_red], color='red', linestyle='--', alpha=0.6,
            label=f'Red max: {Theta_red[idx_red]:.2f} deg')
plt.axvline(h_vals[idx_blue], color='blue', linestyle='--', alpha=0.6,
            label=f'Blue max: {Theta_blue[idx_blue]:.2f} deg')
plt.xlabel('Impact parameter, h')
plt.ylabel(r'Deviation Angle, $\Theta_D$ (deg)')
plt.title('Raindrop Problem: Deviation Angle vs Impact Parameter')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

Red light rainbow angle: 42.39 deg at h = 0.8624 (n = 1.33083)

Blue light rainbow angle: 41.57 deg at h = 0.8594 (n = 1.33650)

Blue light is deviated less than red light.

Raindrop Problem: Deviation Angle vs Impact Parameter

